■ Budget Travel AI

Complete Technical Documentation & System Architecture Guide

Version: 3.0.0 (SPA + Route Optimizer)	**Author / Maintainer:** ojaspaul123
Architecture: FastAPI + Leaflet + OSRM	**Deployment:** Vercel Serverless

1. Executive Summary

Mainstream mapping applications (such as Google Maps or Yelp) prioritize commercial visibility or general popularity, frequently burying budget-friendly eateries, student spots, and affordable lodging beneath high-cost establishments. **Budget Travel AI** solves this by layering deterministic price-tier normalization and rating-weighted scoring over the Google Places API (New), presented through a high-performance single-page web app equipped with an interactive satellite map, nearest-neighbor TSP multi-stop route optimization, and OSRM walking/driving tour calculation.

2. Key Benefits & Value Proposition

- **Strict Budget Filtering:** Five explicit budget tiers (Free, Inexpensive $, Moderate $$, Expensive $$$, Any) ensure results match real travel budgets.
- **Multi-Stop Route Optimizer:** Traveling Salesperson (TSP) nearest-neighbor algorithm orders saved or top spots sequentially to minimize travel distance.
- **Interactive Road & Footpath Routing:** Integrated OSRM routing engine draws real pedestrian walkways and driving routes directly on the Leaflet map.
- **Map Style & Theme Switcher:** Instant switching between Satellite Topo, Carto Dark Night Mode, OpenStreetMap, and World Imagery.
- **15-Min & 30-Min Walk Rings:** Visual walking distance boundary isochrones helping travelers stay within walking radius and eliminate taxi fares.
- **1-Click Multi-Stop Google Maps Navigation:** Direct multi-waypoint deep navigation URLs linking all itinerary stops in sequence.
- **Trip Bucket List:** Client-side trip bookmarking drawer powered by browser localStorage with one-click tour planning.
- **Serverless & Production Ready:** Pre-configured with vercel.json for zero-maintenance Python serverless deployment.

3. System Architecture & Tech Stack

The project follows a decoupled, cloud-ready architecture comprising an asynchronous FastAPI backend and a modern vanilla HTML5/CSS3/JavaScript single-page frontend:

Component	Technology	Key Role & Responsibility
Frontend UI	HTML5 + Vanilla CSS3	Dark glassmorphism design system, responsive cards, slide-out drawer.

Frontend Logic	Vanilla JavaScript (ES6+)	TSP route sequencing, debounced geocoding, filters, localStorage bucket list.
Routing Engine	OSRM Public API (Foot/Drive)	Real road/pedestrian network coordinates and duration calculations.
Geospatial Map	Leaflet.js 1.9.4	Multi-tile switcher, glowing route polylines, numbered waypoint pins.
Base Tiles	Esri Topo / Carto Dark / OSM	Multiple selectable map tile layers with zero key cost.
API Backend	FastAPI (Python 3.10+)	Asynchronous REST endpoints, CORS middleware, Pydantic validation.
External APIs	Google Places & Geocoding	Places API (New) searchText endpoint and address geocoding.
Deployment	Vercel Serverless	@vercel/python serverless API + @vercel/static asset CDN.

4. End-to-End Workflow

1. **Location Resolution:** User enters a query (e.g. 'Patia Bhubaneswar') triggering GET /api/geocode, or activates GPS to detect device latitude and longitude.
2. **Search Query Execution:** User selects category, maximum budget tier, and search radius in km (0.5 to 25.0 km). Frontend posts payload to POST /api/search.
3. **Google Places API Query:** Backend queries Google Places API (New) searchText endpoint requesting a field mask (id, displayName, location, priceLevel, rating, etc.).
4. **Budget Filtering & Scoring:** Backend score_by_budget() normalizes raw prices, filters places exceeding max_budget_level, and sorts candidates by rating and review count.
5. **Geospatial & Route Rendering:** Leaflet initializes map with selectable tile layers, draws the search radius / walking rings, and mounts interactive place pins.
6. **Itinerary & Route Optimization:** User clicks 'Quick Tour' or 'Plan Optimized Tour' — frontend runs TSP nearest-neighbor ordering, queries OSRM for footpath/road geometry, and renders the route with travel metrics and Google Maps export.

5. API Specifications & Contracts

POST /api/search — Core Search & Budget Ranking Endpoint

Request Payload:

```
{ "query": "cafes", "lat": 20.3605, "lng": 85.8248, "radius": 5000, "max_budget_level": 2 }
```

Response Payload (200 OK):

```
{ "count": 1, "results": [ { "id": "places/ChIJ...", "name": "Two Hearts Cafe", "address":
"Patharagadia, Bhubaneswar, Odisha", "rating": 4.9, "user_rating_count": 1420, "price_level":
"Inexpensive", "price_symbol": "$", "price_score": 1, "category": "Cafe", "open_now": true,
"google_maps_url": "https://maps.google.com/...", "location": {"latitude": 20.3612, "longitude":
85.8239} } ] }
```

6. Deployment & Local Execution

Running Locally (Single Command):

```
# 1. Install dependencies pip install -r requirements.txt # 2. Configure .env with your Google Maps API Key echo GOOGLE_MAPS_API_KEY=your_key_here > .env # 3. Run the FastAPI server (Serves both API and Frontend at http://127.0.0.1:8000) python -m uvicorn app.main:app --reload
```

1-Click Vercel Deployment:

1. Push repository to GitHub (`git push origin main`).
2. Import repository on **vercel.com/new**.
3. Under **Environment Variables**, set GOOGLE_MAPS_API_KEY.
4. Click **Deploy**. Vercel automatically builds static assets and mounts serverless Python endpoints.

7. Key Troubleshooting & Engineering Learnings

- **Google Places API 400 Invalid Argument:** Passing PRICE_LEVEL_FREE inside Google's priceLevels request payload causes Google to reject the request. Solution: Query Google without payload price level restrictions and perform deterministic budget scoring in Python via ranker.py.
- **Map Watermark Restrictions:** Public CartoDB tile endpoints introduced API key/referrer requirements. Solution: Default to Esri World Topo Map for crystal clear satellite imagery with zero key requirements.
- **Payload Schema Differences:** Google raw API responses format names as displayName.text. Solution: Implemented normalize_place() to seamlessly support both legacy and normalized schemas.
- **CORS Interoperability:** Configured CORSMiddleware in app/main.py to allow external frontends on Vercel to query the backend securely.

Budget Travel AI • Open Source Project • Created for High-Impact Travel Discovery